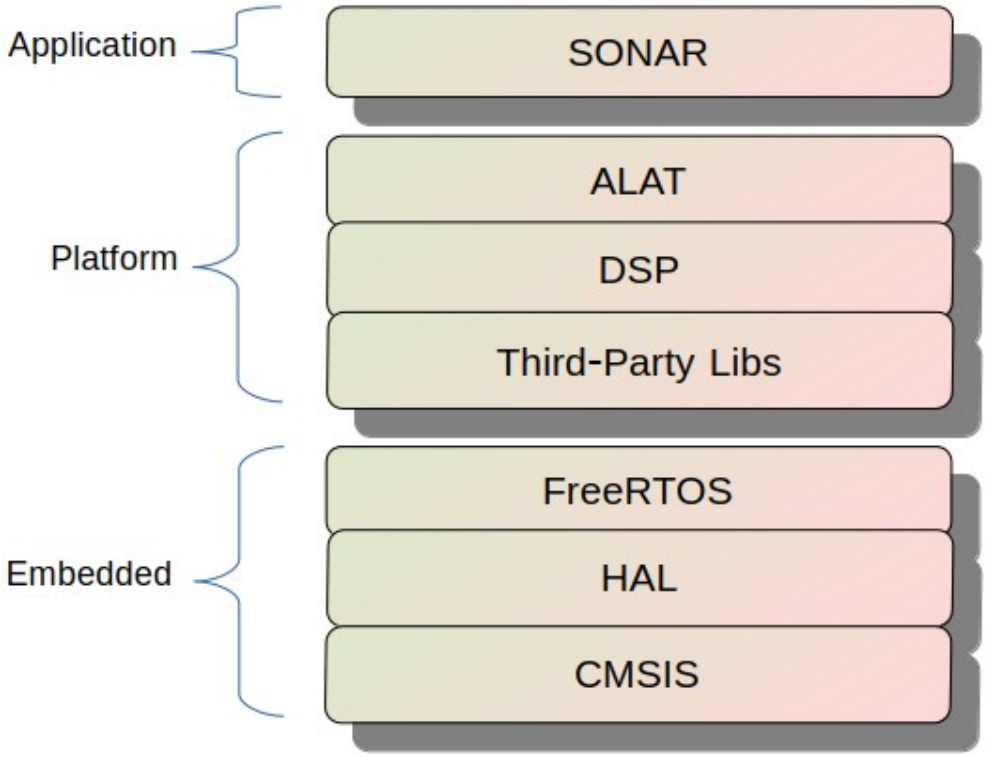
2.2. Yazılım Tasarımı

Proje gereksinimleri karşılayacak tüm yazılımlar yerli olarak tasarlanmış ve kullanılmıştır. Gerekli sinyal işleme fonksiyonlarından son kullanıcı arayüzlerine kadar çeşitli katmalarda ve amaçlarda tüm operasyonlar tarafımız tarafından geliştirilmiştir. Şekil X'te genel yazılım mimarisi gösterilmektedir.



Şekil X. Genel Yazılım Mimarisi

Proje yazılımları üç katmanda incelenebilir. Gömülü katmanda, kullanılan mikrodenetleyici için yazılmış olan program bulunmaktadır. Platform katmanında ise uygulama için gerekli kütüphaneler yer almaktadır. Burada hazır sistem kütüphanelerinin yanında iki kütüphane yerli olarak tasarlanmış. Bunlar, DSP (*Digital Signal Processing*) ve ALAT (*Advanced Linear Algebra Toolkit*) kütüphaneleridir. Sonraki bölümde bu kütüphaneler detaylı şekilde incelenmiştir.

Yazılımların geliştirildiği ortam özellikleri ise şunlardır:

- İşletim Sistemi: Ubuntu 24.04.4 LTS, 64-bit
- Masaüstü Ortamı: GNOME 46
- Pencereleme Sistemi: Wayland
- Çekirdek (*Kernel*) versiyonu: Linux 6.8.0-63

2.2.1. Kütüphaneler

Proje kapsamı gereği birçok farklı kütüphane kullanılmıştır. Kullanılan kütüphanleri şu şekilde sıralayabiliriz:

- CMSIS (Cortex-M soyutlama için)
- STM32 Cube HAL (donanım soyutlama için)
- FreeRTOS (gerçek zamanlı işletim sistemi için)

- GNU C (standart POSIX uyumlu API için)
- SQLite3 (veri tabanı entegrasyonu için)
- Shumate (GPS haritalama için)
- Cairo (2D görselleştirme için)
- GTK4/Adwaita (kullanıcı arayüzü uygulaması için)
- GNU GSL (numerik hesaplamalar için)
- DSP (sonraki bölümde açıklanmıştır)
- ALAT (sonraki bölümde açıklanmıştır)

Burada bizim tarafımızdan tasarlanan spesifik iki kütüphaneyi inceleyelim.

2.2.1.1. Dijital Sinyal İşleme Kütüphanesi

Projenin en önemli fonksiyonu mikrofon verilerinin işlenmesi ve yorumlanmasıdır. Bunu oluşturduğumuz sinyal işleme kütüphanesi ile yapmaktayız. Hazır bir kütüphane kullanmamamızın en önemli sebebi, projede bazı özel algoritmalara gereksinimleridir. Bu kütüphane ayrıca deterministik karakteristiğe sahiptir. Şekil X'te ve Şekil X'te örnek bir kullanım görmekteyiz.

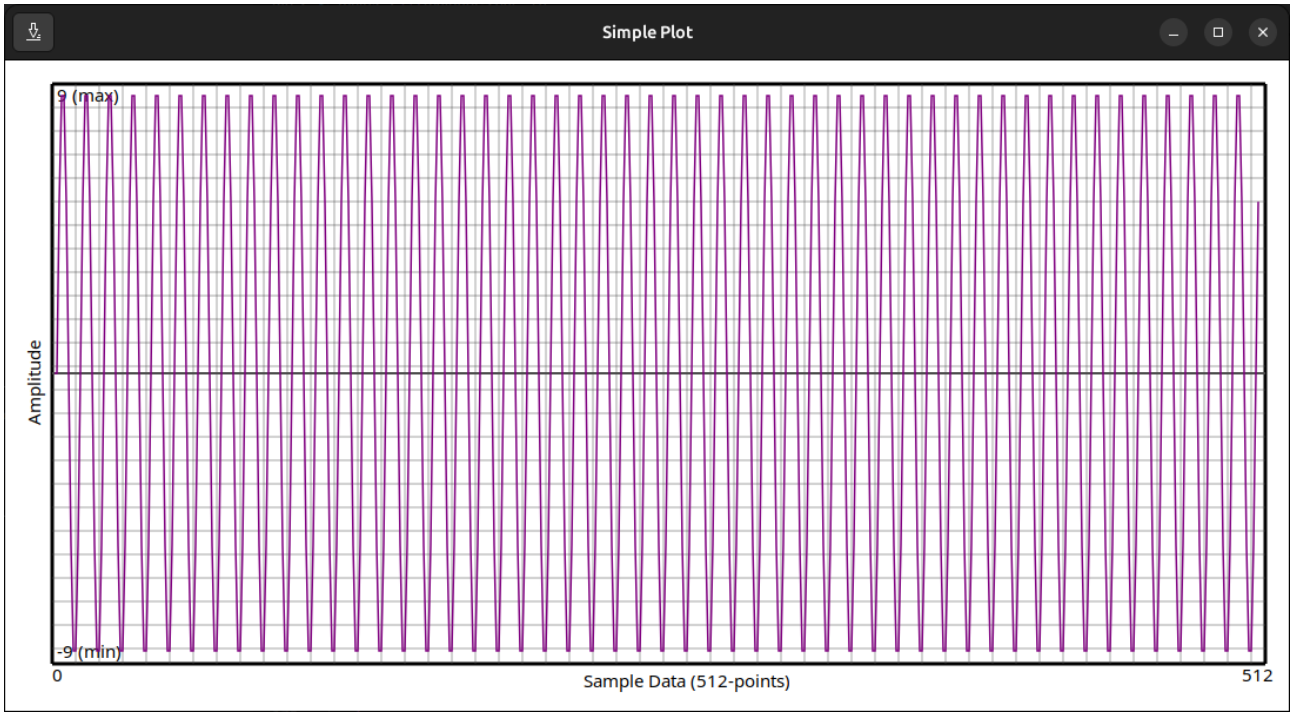
```
DspTime sample;   /* time-domain object defined in stack */
DspPlot plot;     /* plotting object defined in stack */

dsp_signal_sin(10, 100, 1000, 0, 512, &sample); /* sin signal */

plot.title = "Simple Plot";
plot.sample = &sample;
plot.color = DSP_COLOR_PURPLE;
plot.factor = 2;
plot.width = 1.5;

dsp_plot_sample(&plot);    /* plot the time domain signal */
```

Şekil X. Sinüs Sinyalinin Üretilmesi



Şekil X. Sinüs Sinyalinin Görselleştirilmesi

Sinyal işleme kütüphanesi şu modüllerden oluşmaktadır:

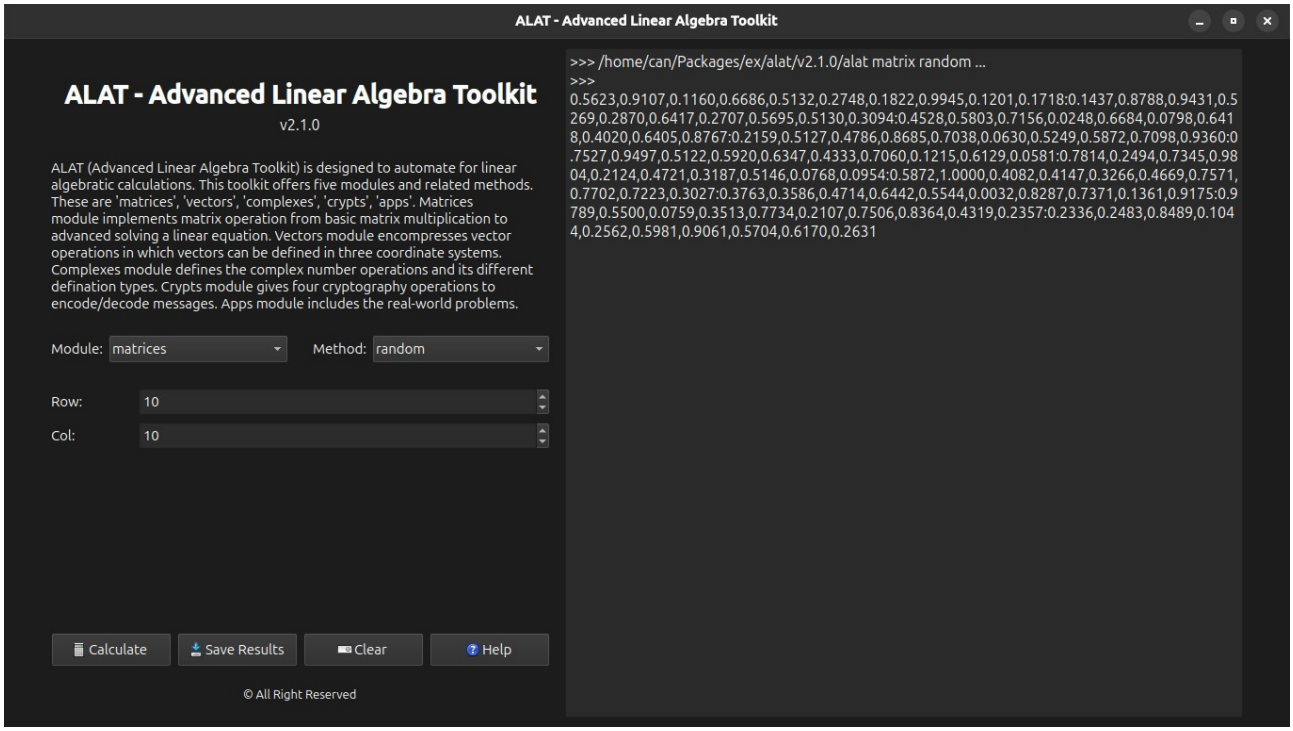
- *core*
- *filter*
- *signal*
- *transform*
- *window*
- *plot*
- *arrival*
- *beamform*

Burada *core* modülü temel zaman-frekans domeni analizlerini, *filter* modülü FIR ve IIR temelli filtre tasarımlarını, *signal* modülü analizlerde sıklıkla kullanılan sinyal formlarının üretilmesini, *transform* modülü zaman-frekans domeni dönüşümünü sağlayan Fourier denklemlerini, *window* modülü pencereleme algoritmalarını, *plot* modülü zaman domeni sinyal görselleştirilmesinin, *arrival* modülü geliş açısı/yönü hesaplamalarını ve *beamform* modülü ise *beamforming* algoritmalarını içermektedir.

Bu sinyal işleme kütüphanesine buradan ulaşabilirsiniz.

2.2.1.2. Lineer Cebir Kütüphanesi

Lineer cebir operasyonları için tasarladığımız bu kütüphane ile projede ihtiyaç duyulan nümerik hesaplamaların gerçekleştirilmesi hedeflenmektedir. Bu kütüphane, projeden bağımsız olarak ve çok daha önceden geliştirmeye başlanmıştır. Fakat sonradan bu projeye dahil edilmiştir. Önceki bahsettiğimiz sinyal işleme kütüphanesinin aksine, bu kütüphane yer istasyonuna yardımcı olarak ikincil bir program şeklinde kullanılmalıdır. Şekil X'te örnek bir kullanım görmekteyiz.



Şekil X. Linear Cebir Kütüphanesinin Arayüzü

Bu kütüphane şu modülleri içermektedir:

- *matrices*

- *vectors*
- *complexes*
- *crypts*
- *apps*

Burada *matrices* modülü matriks operasyonlarını, *vectors* modülü farklı koordinat sistemlerini destekleyen vektör operasyonlarını, *complexes* modülü kompleks sayı operasyonlarını, *crypts* modülü temel kriptoloji operasyonlarını ve *apps* modülü ile bazı gerçek-hayat problemlerin çözümünü bulundurmaktadır.

Bu lineer cebir kütüphanesine buradan ulaşabilirsiniz.

2.2.2. Gömülü Yazılım

Gömülü yazılım STM32H7x mikrodenetleyici tabanlı elektronik kart için FreeRTOS çekirdeği ile geliştirilmiştir. Bu yazılımın temel görevi, elektronik kart üzerinde bulunan tüm komponentlerin eş güdüm halinde çalıştırmaktır. Gömülü yazılımın kontrol ettiği komponentler şunlardır:

- MEMS mikrofonlar
- IMU sensörü
- GPS modülü
- LoRa modülü
- SD kart

Burada MEMS mikrofonlar, elektronik kart etrafındaki ses verilerinin toplanması; IMU sensörü, kartın hızlanmasını ve rotasyonunu tespit etmek; GPS modülü, kartın mevcut konumu bulmak; LoRa modülü, kablosuz haberleşmeyi sağlamak; SD kart ise kritik verilerin depolamak için kullanılmaktadır.

MEMS mikrofonlar, mikrodenetleyicinin DFSDM (*Digital Filter for Sigma-Delta Modulator*) periferi üzerinden 12kHz frekans değeri ile örneklenmektedir. Elektronik kart üzerinde toplam sekiz adet mikrofon bulunduğu için gömülü yazılımda, dört görev (*task*) tasarlanmıştır. Her görev, iki adet mikrofonu okumaktadır. DFSDM periferinden okunan her veri 32-bit uzunluğundadır. Fakat gömülü yazılım, bu verilerin 8-bit MSB (*Most Significant Bit*) olacak şekilde kaydırır.

IMU sensörü, 6-eksenli olup ivmeölçer, jiroskop ve sıcaklık ölçer bulundurmaktadır. İvmeölçer, 416Hz/16g değeri ile ölçeklendirilmiş olup ürettiği veriler m/s^2 olacak şekilde paketlenir. Jiroskop ise 416Hz/2000dps değeri ile ölçeklendirilmiş ve üretilen veriler dps (*degrees per second*) olarak paketlenir. Gömülü yazılım, bu üretilen verileri SPI (*Serial Peripheral Interface*) periferi üzerinde okumaktadır.

GPS modülü, UART (*Universal Asynchronous Receiver Transmitter*) periferi üzerinden NMEA cümlelerini üretmektedir. Gömülü yazılım bu ham NMEA verilerini alıp gerekli çıktıları bu ham verileri işleyerek oluşturmaktadır. Bu gerekli çıktılar, UTC zaman, enlem, boylam, kalite, kullanılan uydu sayısı, yükseklik, statü, hız (knots), kapsam (degrees) ve tarihtir.

LoRa modülü ise şimdiye kadar bahsettiğimiz verileri kablosuz olarak yer istasyonuna iletmek için kullanır. Bu projede iki adet LoRa modülü bulunmaktadır. Birincisi gömülü yazılımın verileri yazdığı verici LoRa modülüdür. Gömülü yazılım UART periferi üzerinden verilerin iletimini başlatır. Alıcı LoRa ise yer istasyonu tarafında bulunur ve yer istasyonu, UART-USB çevirici üzerinden iletilen verileri belirli bir zaman aşımı ile okumaktadır. Daha sonra bahsedeceğimiz gibi okunan bu veriler yer istasyonunda gerekli şekilde işlenir.

SD kart ile ayrıca gönderilen bu veriler elektronik karta kaydedilir. Bu özellikle iletişimi koptuğu sahalarda ve kritik sistemlerde kullanılan bir yöntemdir. Öncelikle SD kart yer istasyonunda *ext4* dosya sistemi ile formatlanmaktadır. Ardından elektronik karta takılıp blok halinde yazma işlemi belirli bir zaman aşımı tamamlanır.

2.2.2.1. Donanım Soyutlama Katmanı

Herhangi bir gömülü yazılımı geliştirmek için donanım soyutlama katmanı kütüphaneleri gerekir. STM32H7x mikrodenetleyicisi için bu kütüphaneler CMSIS (*Common Microcontroller Software Interface Standard*) ve STM32H7 HAL (*Hardware Abstraction Layer*)'dır.

CMSIS kütüphanesi, ARM tarafından Cortex-M için geliştirilmiştir. Bu kütüphane Cortex-M içerisinde bulunan donanımları soyutlamak için kullanılır. STM32 HAL kütüphanesi ise, ST tarafından geliştirilen ve mikrodenetleyicide bulunan tüm periferlerin soyutlanması için tasarlanmıştır. Bu iki kütüphane her zaman birlikte kullanılmaktadır.

2.2.2.2. FreeRTOS Çekirdeği

Görüleceği gibi gömülü yazılımın yapması gerek birçok görev bulunmaktadır. Bu görevleri eş yordam halinde gerçekleştirmek için gerçek zamanlı işletim sistemi olan FreeRTOS çekirdeği kullanılmıştır. Donanım soyutlama katmanının üzerinde bulunun bu çekirdek daha önce bahsettiğimiz görevleri oluşturur ve çalıştırır.

Tanımlanan tüm bu görevler, atanan öncelik katsayısı ve bellek alanına göre sırasıyla çalıştırılır. Yüksek öncelik katsayısı olan görevler, daha düşük öncelik katsayısına sahip olan görevlere göre her zaman öncelenir.

Bu tür kütüphanelerin sağladığı en önemli fonksiyon, bağlam değiştirmedir (*context switch*). Bu özellik ile tüm görevler belirli bir zaman aralığında, çoğunlukla konfigürayon dosyaları üzerinde belirlenir, görevlerin sırasıyla çalıştırılması sağlanır ve böylece her görev eşit bir zaman aralığına sahip olur.

Gömülü yazılıma buradan ulaşabilirsiniz.

2.2.3. Yer İstasyonu

Yer istasyonu son kullanıcı için tasarlanmış ve şimdiye kadar bahsedilen tüm süreçlerin yönetimini sağlamaktadır. Birçok modülü bulunan bu yer istasyonunun temel görevi yük verisini haberleşme kanalından okuyup gerekli şekilde işleyerek son kullanıcıya sunmaktır. Şimdi bu modüllere ve görevlerine bakalım.

2.2.3.1. *Microphone* Modülü

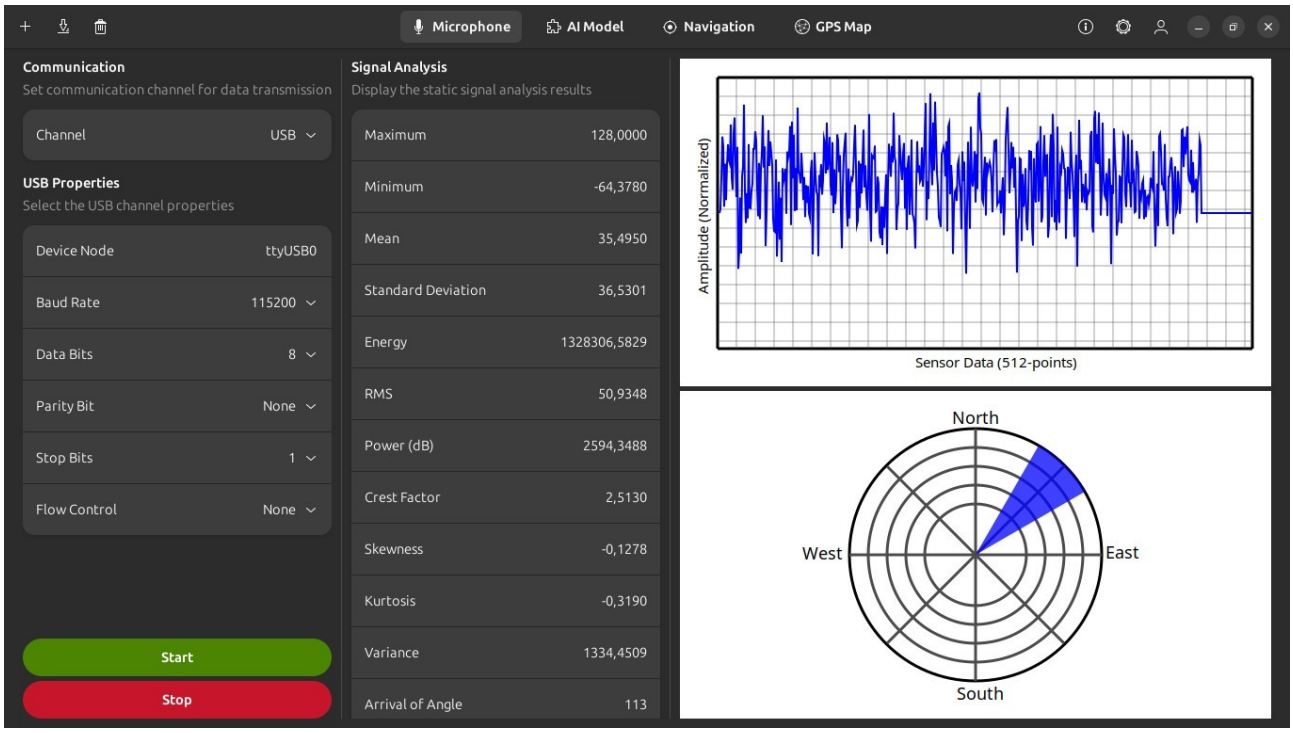
Bu modül, yük verisinde bulunan mikrofon sensör verisi okuyarak birtakım operasyonlar gerçekleştirir. Öncelikle bu modülde haberleşme kanalını ve ilgili parametrelerinin seçilmesinin sağlayan kontrol paneli bulunmaktadır. Haberleşme kanalı okunmaya başlandıktan sonra belirli bir zaman aşımı ile mikrofon verileri ile bazı sinyal işleme operasyonları gerçekleştirilir. Bu daha önce de bahsettiğimiz ve yerli olarak geliştirdiğimiz sinyal işleme kütüphanesi ile yapılır. Sonuçlar numerik olarak arayüzde orta kısımda gösterilir.

Ayrıca bu işlenen mikrofon verileri iki farklı parselde görselleştirilir. Bunlardan birincisi işlenen verilerin genliğini gösteren sağ üstteki parseldir. İkinci parselde ise en yüksek ses yoğunluğa sahip mikrofon gösterilmektedir.

Bahsettiğimiz sinyal işleme süreçlerini şu şekilde özetleyebiliriz:

- Mikrofon verilerinin hazırlanır.
- Dominant frekans değeri bulunur.
- Geliş açısı/yönü algoritmaları uygulanır.
- *Beamforming* algoritmalarının uygulanır.
- Diğer istatistiksel analizlerin yapılır.
- Modül arayüzü güncellenir.

Yer istasyonunda üç farklı haberleşme kanalı desteklenmiştir. Bunlar UART, USB, Wi-Fi'dir. Her kanal kendi parametrelerine sahiptir. Bu parametrelerin ayarlanması ile kanal okuma operasyonu başlatılır. Şekil X'te bu modülün arayüzünü görmekteyiz.



Şekil X. "Microphone" Modülü Arayüzü

2.2.3.2. *AI Model* Modülü

Bu modül, klasik frekans-tabanlı hedef teşhis yöntemlerinin yanında yapay zeka destekli bir alternatif sunmaktadır. Bu amaçla bir derin öğrenme modeli geliştirmiştir. Geliştirilen bu modelin eğitim, değerlendirilmesi, ve testi arayüz üzerinden gerçekleştirilir.

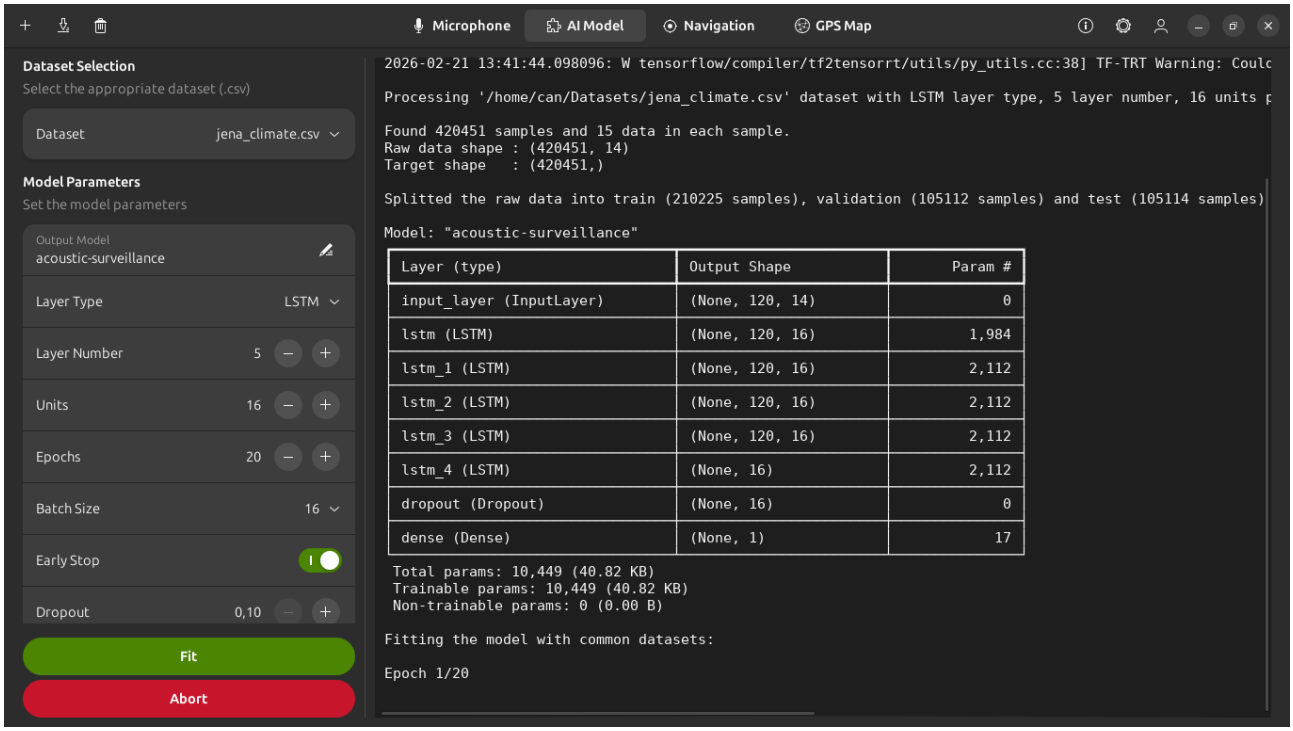
Arayüz üzerinde sol tarafta eğitilecek veri setinin ve ilgili model parametrelerinin seçilmesi için bir panel bulunmaktadır. Sağ tarafta ise model sonuçları belirli bir zaman aşımı ile gösterilmektedir.

Bu derin öğrenme modeli RNN (*Recurrent Neural Network*) mimarisine dayalıdır. Bu seçimin temel nedeni mikrofon verilerinin zaman bağımlılığı karakteristiğidir. Bu literatürde zaman-verisi olarak isimlendirilir. RNN mimarileri bu tür verileri işlemek için tasarlanmıştır.

Geliştirdiğimiz bu derin öğrenme modelinin çalışma süreçlerini şu şekilde özetleyebiliriz:

- Veri setinden veriler yüklenir.
- Verilerin eğitim, değerlendirme ve test gruplarına bölünür.
- RNN modelinin ilgili parametreleri ile tasarlanır.
- Tasarlanan modelin derlenir.
- Derlenen modelin eğitilir.
- Eğitim sonucunda modelin doğruluğunun ölçülür.
- Test verileri ile modelin test edilir.

Başlangıç olarak kullandığımız veri seti zaman-serisi karakteristiğine sahip bir hava durumu verisidir. Projemiz özelinde kullanabileceğimiz açık kaynak bir veri seti bulunmamaktadır. Fakat sonraki bölümde de anlattığımız gibi, yer istasyonunda veri tabanı entegrasyonu bulundurmaktadır. Haberleşme kanalından okunan veriler belirli bir zaman aşımı ile bu veri tabanına kaydedilir. Derin öğrenme modeli için gerekli miktarda ve büyüklükte veri oluşturulması hedeflenmektedir. Şekil X'te bu modülün arayüzünü görmekteyiz.

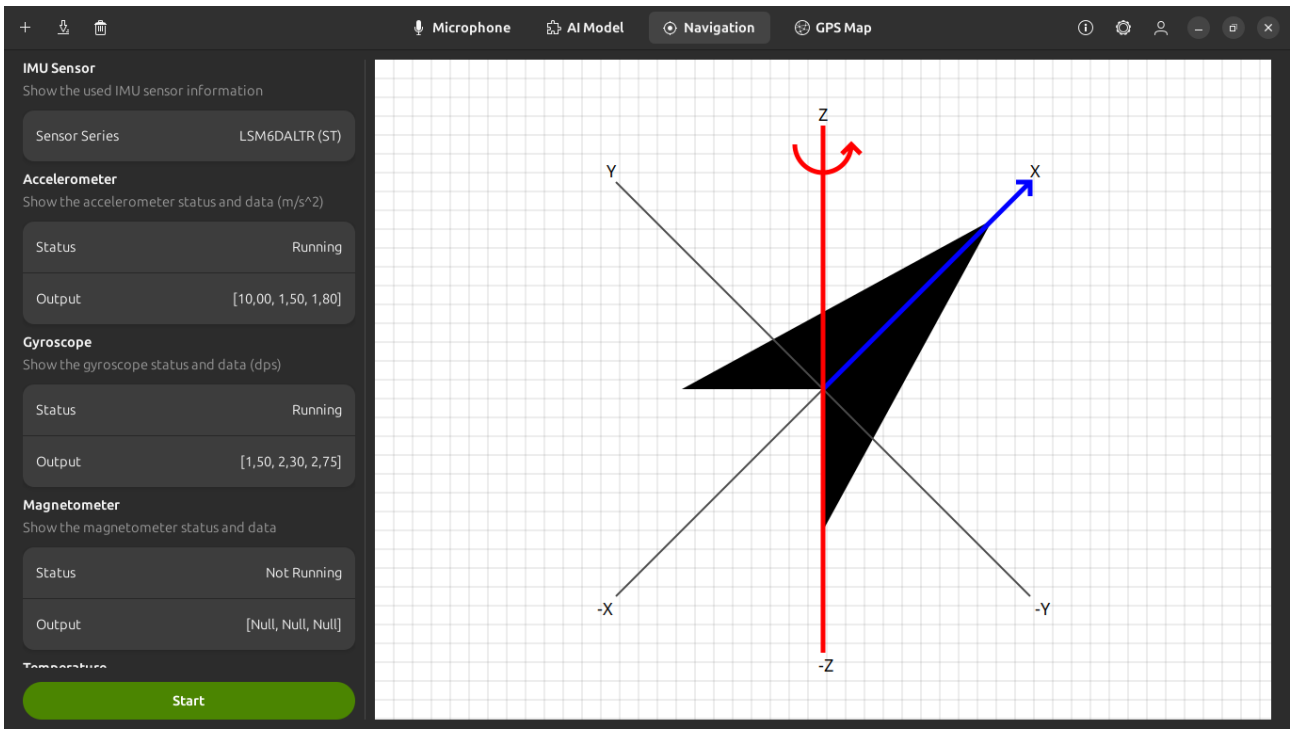


Şekil X. "AI Model" Modülü Arayüzü

2.2.3.3. *Navigation* Modülü

Bu modül, sistemde bulunan IMU (*Inertial Measurement Unit*) sensör verisini kullanarak sistemin fiziksel hızlanmasını ve rotasyonunu hem numerik olarak göstermekte hem de bir parsel üzerinde görselleştirilmektedir. Ayrıca kullandığımız bu sensör sıcaklık verisi de üretmektedir.

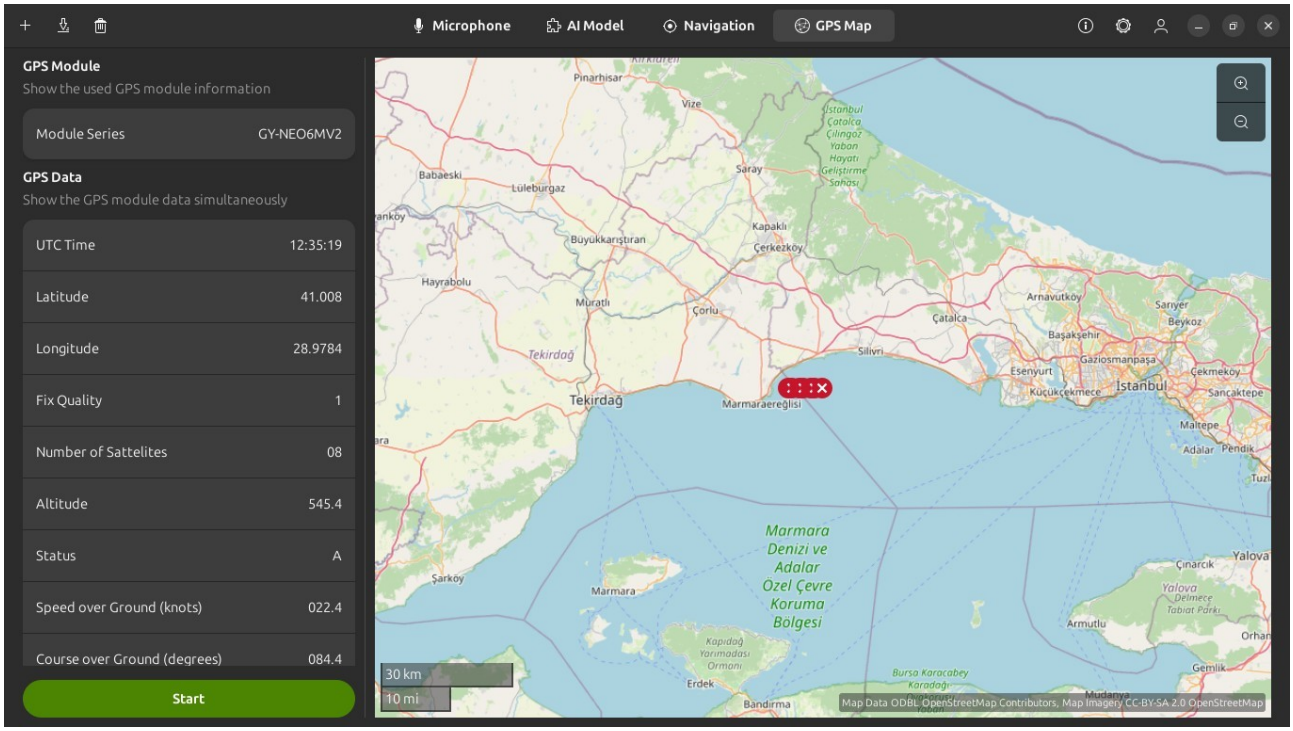
Arayüz üzerinde sol tarafta numerik olarak gösterilmektedir. Sağ tarafta ise hızlanma, mavi bir ok ve rotasyon, kırmızı bir çubuk şeklinde gösterilir. Şekil X'te bu modülün arayüzü gösterilmektedir.



Şekil X. “Navigation” Modülü Arayüzü

2.2.3.4. *GPS Map* Modülü

Bu modül, sistemde bulunan GPS modülünün ürettiği veriler gösterilmektedir. Ayrıca okunan enlem-boylam verileri gerçek zamanlı bir harita üzerinde belirli bir zaman aşımı ile işaretlenmektedir. Böylece sistemin harita üzerinde hareketini görülebilir. Şekil X’te bu modülün arayüzü gösterilmektedir.



Şekil X. “GPS Map” Modülü Arayüzü

2.2.3.5. Alt Sistemler

Yer istasyonu, daha önce bahsettiğimiz modüllerin çalıştırılması ve tüm sistemi sorunsuz bir şekilde yürütülmesi için birçok alt sistem barındırmaktadır. Bunlardan iki tanesini, veri tabanı entegrasyonu ve günlük kayıt tutma (*logging*) sistemleridir.

Haberleşme kanalından okunan veriler sinyal işleme adımlarının uygulanmasının ardından veri tabanına kaydedilmektedir. Bu özellikle derin öğrenme modeli için gereken verilerin toplanması ve depolanması için kullanılacaktır.

Diğer bir sistem olan günlük kayıt tutma sisteminde ise yer istasyonunda gerçekleştirilen önemli çıktılar ve mesajlar *.log* dosyalarına belirli bir formatta kaydedilmektedir. Bu özellikle geriye doğru sistem taraması ve hata giderme için kullanılmaktadır.

Yer istasyonuna buradan ulaşabilirsiniz.

2.2.4. Güvenlik Standartları

Özellikle kritik sistem yazılımların geliştirilmesinde birtakım endüstriyel standartlar bulunmaktadır. Tüm profesyonel ve ticari yazılımlar bu standartlar doğrultusunda gerçekleştirilir. Bu projemizde tüm gömülü/platform/uygulama yazılımları MISRA C:2012 kurallarına göre yazılmıştır.